AUTONOMOUS ENGINEERING ARCHITECTURE

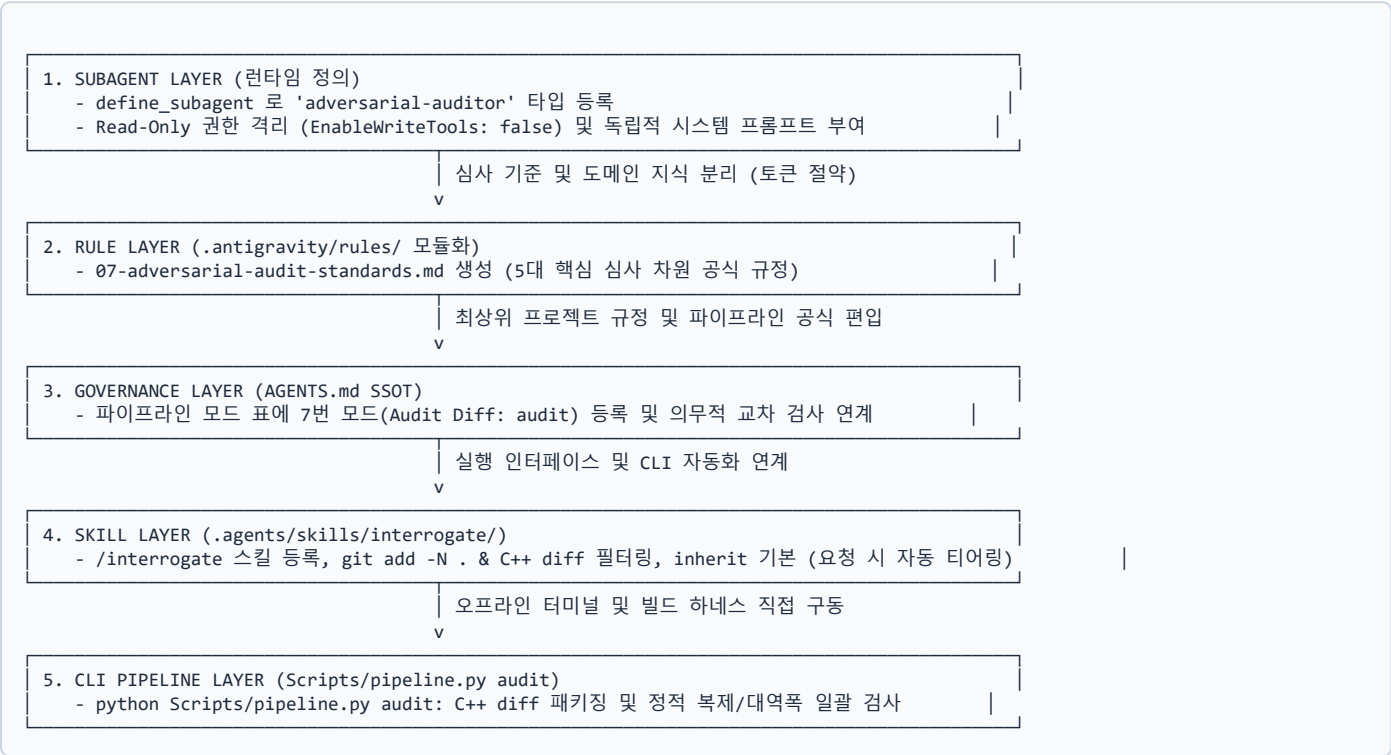
SubAgent 추가에 따른 시스템 연쇄 영향도 분석 (Rule, AGENTS.md, Skill, Pipeline 통합)

독립 서브에이전트 도입 시 프로젝트 거버넌스, 지침, 룰, 실행 인터페이스, CLI 파이프라인의 5대 계층 연쇄 분석

프로젝트	문서 성격	발행 형식	일자
FC (Unreal Engine 5.8)	아키텍처 연쇄 수정 영향도 분석 보고서	Executive PDF Edition (최종 갱신)	2026. 10. 28

Executive Summary
Antigravity 에이전트 환경에서 **새로운 SubAgent(`adversarial-auditor`)**를 추가하는 작업은 단순한 프롬프트 정의에 그치지 않습니다. 에이전트 간의 권한 분리, 엄격한 심사 루브릭, 자율 파이프라인 연계, 슬래시 명령어 트리거를 유기적으로 완성하기 위해 ① **Rule 계층**, ② **거버넌스 계층(AGENTS.md)**, ③ **Skill 계층**, ④ **CLI 파이프라인(pipeline.py audit)**이 반드시 동기화되어 수정되어야 합니다. 본 보고서는 서브에이전트 도입 시 이 5대 계층이 어떻게 상호작용하고 변경되었는지를 체계적으로 규명합니다.

1. SubAgent 추가 시 5대 계층 연쇄 변경 개요 (5-Tier Co-Evolution)



2. 각 계층별 세부 변경 내역 및 설계 의도

2.1 SubAgent Layer (정의 및 격리)

- **변경 내용**: `define_subagent` API를 통해 `adversarial-auditor` 등록.
- **핵심 설계 의도 (Read-Only)**: `EnableWriteTools: false`를 강제하여, 심사 서브에이전트가 코드를 직접 수정하는 과정에서 발생할 수 있는 환각 및 2차 버그 유발을 원천 차단했습니다.
- **모델 다변화**: `Model: pro`를 지원하여, 코드를 작성한 모델보다 더 깊은 추론력을 가진 상위 모델이 비판적 결함을 찾아내도록 구성.

2.2 Rule Layer: `.antigravity/rules/` 의 확장

- **신규 추가 파일**: `.antigravity/rules/07-adversarial-audit-standards.md`
- **도메인 룰 분리의 당위성**: 심사 기준 전체를 시스템 프롬프트에 상시 유지하면 서브에이전트 구동 시마다 막대한 컨텍스트 토큰이 소모됩니다. 기준을 외부 룰 파일로 모듈화하면 필요할 때만 참조할 수 있으며, 개발자와 에이전트 모두가 동일한 단일 진실 공급원(SSOT)을 공유합니다.
- **5대 심사 차원의 확립**:
  1. **코드 간결성**: 불필요한 보일러플레이트 및 과도한 클래스 래핑 배제.
  2. **기존 구조 및 코드 활용**: 기존 Subsystem, DataTable, Component 재사용 강제.
  3. **범위 내 적정 확장성**: 기획서에 명시된 범위를 초과하는 미래 대비 과설계(YAGNI 위반) 억제.
  4. **네트워크 최적화**: HasAuthority, RPC Connection 소유권, 복제 조건(COND) 엄수.
  5. **일반 성능 최적화**: Tick 기본 비활성화, 핫패스 힙 할당 제로화, TObjectPtr GC 위생.

2.3 Governance Layer: `AGENTS.md` 의 동기화

- **수정 위치**: `AGENTS.md` Section 3(파이프라인 실행 모드) 및 Section 4(도메인 룰 목록).
- **변경 내역**:

```
┌──────────────────────────────────────────────────────────────────────┐
│                          PIPELINE EXECUTION MODES                    │
├──────────────────────────────────────────────────────────────────────┤
│ 1. [Default] Fast Mode        : Direct C++ coding & Q&A (Zero script overhead)       │
│ ...                                                                  │
│ 7. Audit Diff (`audit`)       : Package C++ diff & static checks for SubAgent review │
│ 8. Full Release (`full`)      : Sequential (Static -> Build -> Test -> [Auto-Commit])│
└──────────────────────────────────────────────────────────────────────┘
```

- **설계 의도**: 세션이 새로 열려도 메인 에이전트가 코드 커밋 전 교차 검증 절차를 자율적으로 기억하고 수행하도록 최상위 헌장에 강제했습니다.

2.4 Skill Layer: `.agents/skills/interrogate/` 의 구축 및 고도화

- **환경 점검 후 고도화된 기능**:
  1. **신규 Untracked C++ 자동 편입**: `git add -N .`을 사전 실행하여 새로 생성된 파일이 diff에서 누락되는 맹점 해소.
  2. **바이너리 에셋 오염 방지**: `git diff HEAD -- "*.h" "*.cpp" "*.cs"`를 통해 `.uasset` 바이너리 diff를 배제하고 토큰을 절약.
  3. **규모별 모델 티어링**: 단순 함수 수정은 `flash`, 복제/GAS/아키텍처 변경은 `pro`를 선택하도록 안내.
  4. **자율 치유(Closed-Loop)**: BLOCKING 판정 시 즉시 메인 에이전트가 자동 패치 적용 후 UBT 빌드 검증 연동.

2.5 Pipeline Layer: `Scripts/pipeline.py audit` 신설

- 터미널에서 `python Scripts/pipeline.py audit`을 입력하면 C++ diff 라인수와 변경 파일 목록을 계산하고, 복제/대역폭 정적 분석을 0.3초 만에 일괄 수행하여 서브에이전트용 패키지를 정형화합니다.

3. 서브에이전트 추가에 따른 종합 시스템 영향도 매트릭스

계층 / 파일	유형	주요 변경 및 추가 내용	상호작용 및 파급 효과
`define_subagent (adversarial-auditor)`	SubAgent	Read-Only 심사 전문 에이전트 런타임 정의, 엄격한 심사관 시스템 프롬프트 부여	코드 오염 없이 비판적 결함만 탐지하는 독립 실행 주체 생성
`.antigravity/rules/07-adversarial-audit.md`	Rule	간결성, 기존 구조 활용, 범위 내 확장성, 네트워크/일반 최적화 5대 루브릭 규정	심사관과 작업자가 동일한 잣대로 소통할 수 있는 객관적 기준 제공
`AGENTS.md (Section 3 & 4)`	Governance	파이프라인 실행 모드 7번 편입, 도메인 룰 목록 링크, 최소 수정 원칙 준수 강조	메인 에이전트가 코드 커밋 전 서브에이전트 감사를 의무화하도록 강제
`.agents/skills/interrogate/SKILL.md`	Skill	/interrogate 슬래시 커맨드, git add -N ., C++ 필터링, inherit 기본 & 요청 시 자동 티어링	원클릭으로 바이너리 오염 없는 순수 C++ diff 교차 검증 절차 구동
`Scripts/pipeline.py (audit mode)`	Pipeline	python Scripts/pipeline.py audit 실행 시 C++ diff 패키징 및 정적 복제 검사 병합	CLI 환경에서도 동일한 감사 패키지를 즉시 추출 가능

4. 엔드투엔드 실행 라이프사이클 (End-to-End Operational Lifecycle)



5. 결론: 상용 수준의 자율 품질 거버넌스 완성

서브에이전트는 독립된 스크립트가 아니라 **[룰 - 거버넌스 - 스킬 - CLI 파이프라인 - 런타임]**의 5중 결합체로 구성될 때 비로소 사람의 개입 없이도 누락 없이 완벽히 작동합니다. FC 프로젝트는 이 체계를 통해 AAA급 언리얼 엔진 5.8 멀티플레이어 환경에서 최고의 개발 안전성을 확보했습니다.